

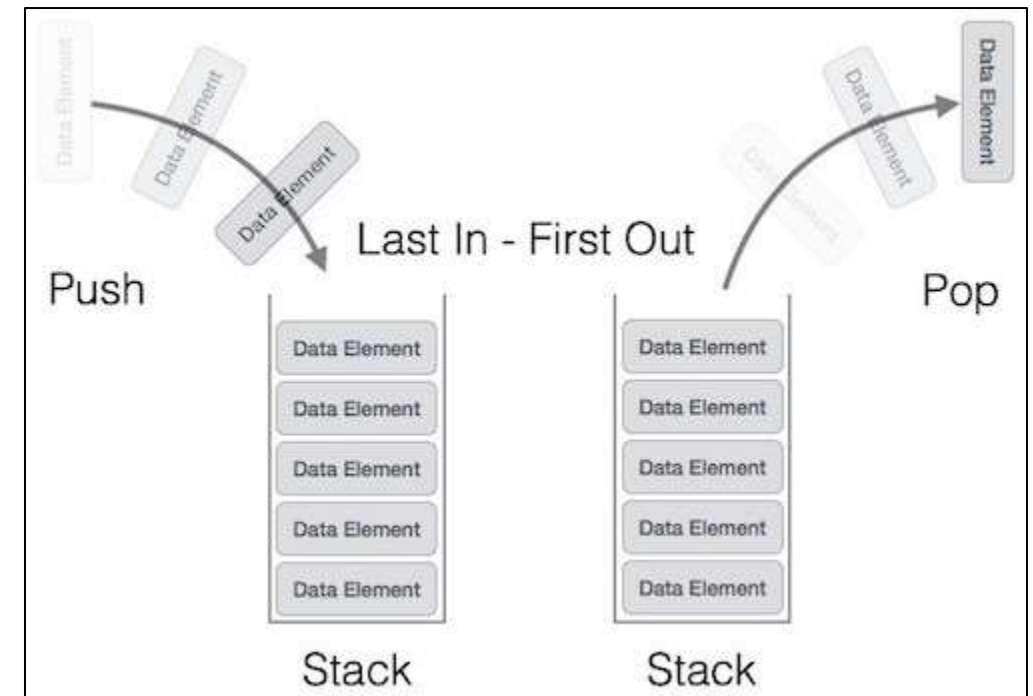
Structures de données linéaires:

ADT Stack

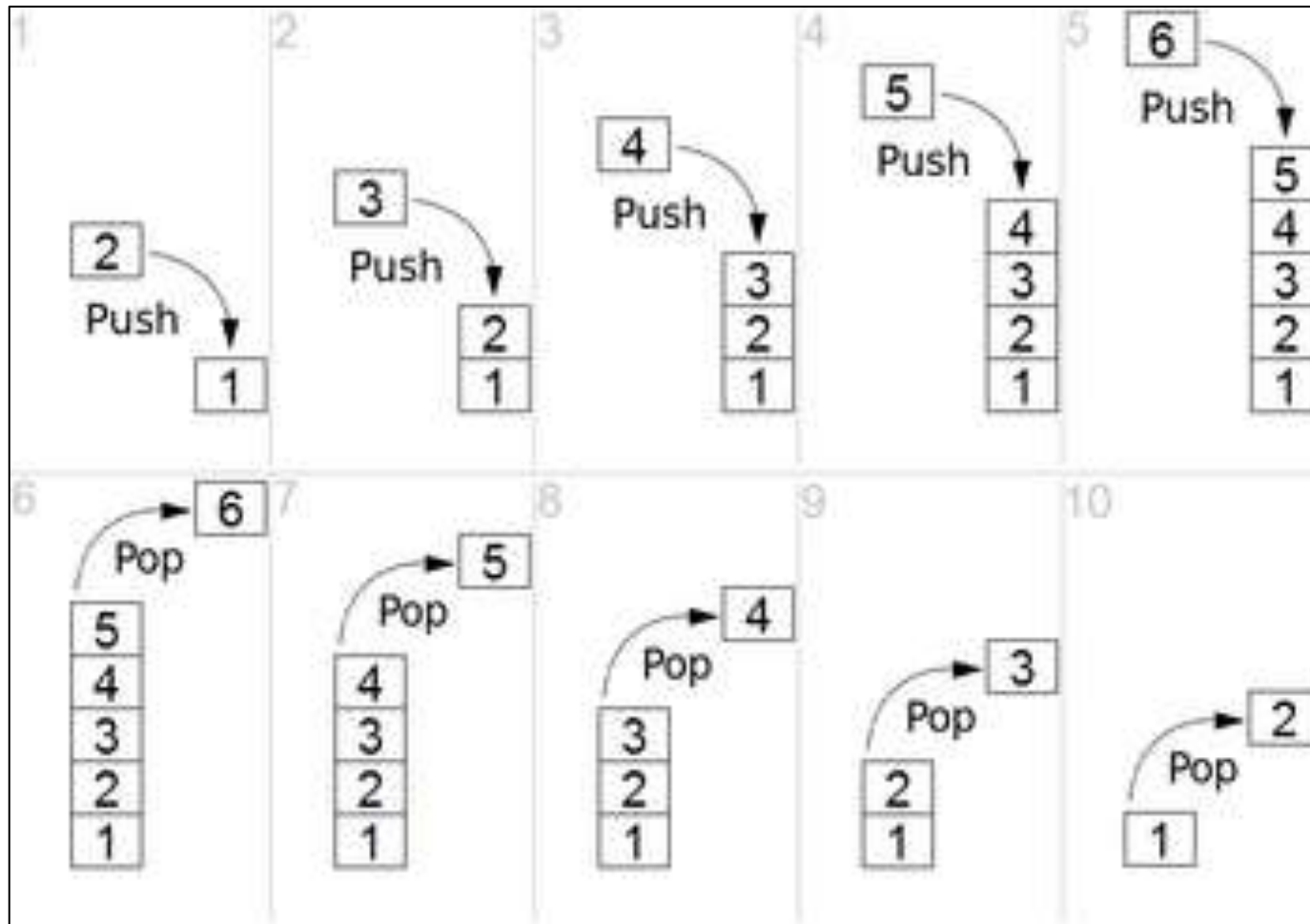
rouanehm@gmail.com

Notion de Stack (pile)

- Une pile est une structure linéaire permettant de stocker et de restaurer des données selon un ordre LIFO (Last In, First Out ou « dernier entré, premier sorti »)
- Les insertions (empilements) et les suppressions (dépilements) sont restreintes à une extrémité appelée sommet de la pile.

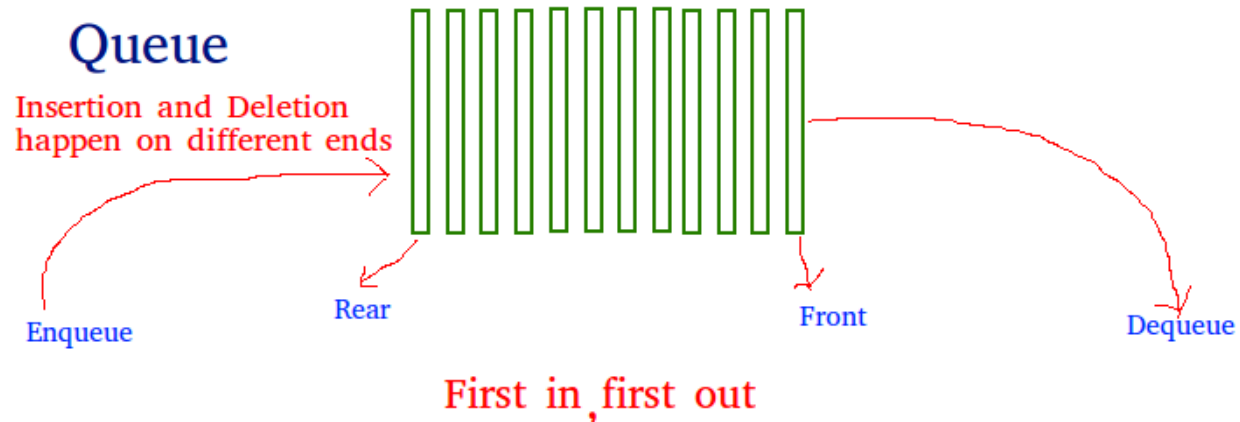
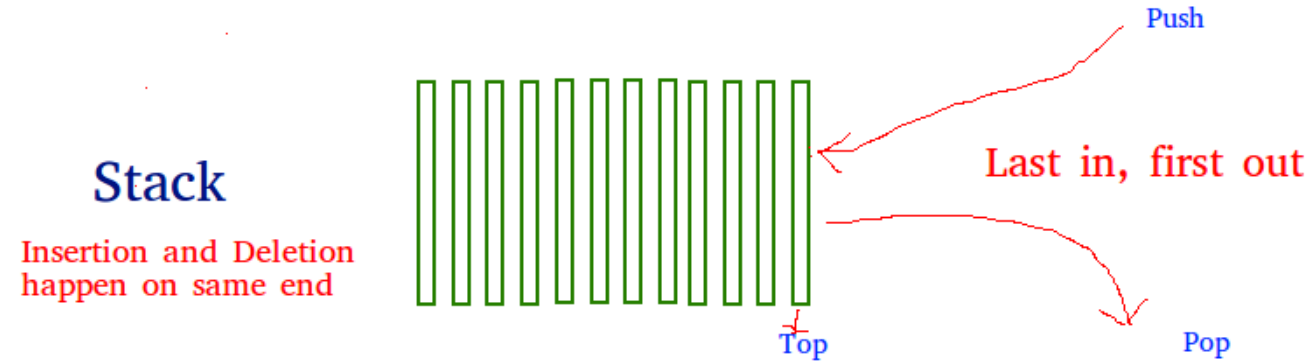


Exemple d'utilisation d'un Stack



[https://en.wikipedia.org/wiki/Stack_\(abstract_data_type\)](https://en.wikipedia.org/wiki/Stack_(abstract_data_type))

Stack versus Queue



Opérations sur un Stack

- **push(o)**: Inserts object *o* onto top of stack
Input: Object; *Output*: none
- **pop()**: Removes the top object of stack and returns it; if stack is empty an error occurs
Input: none; *Output*: Object

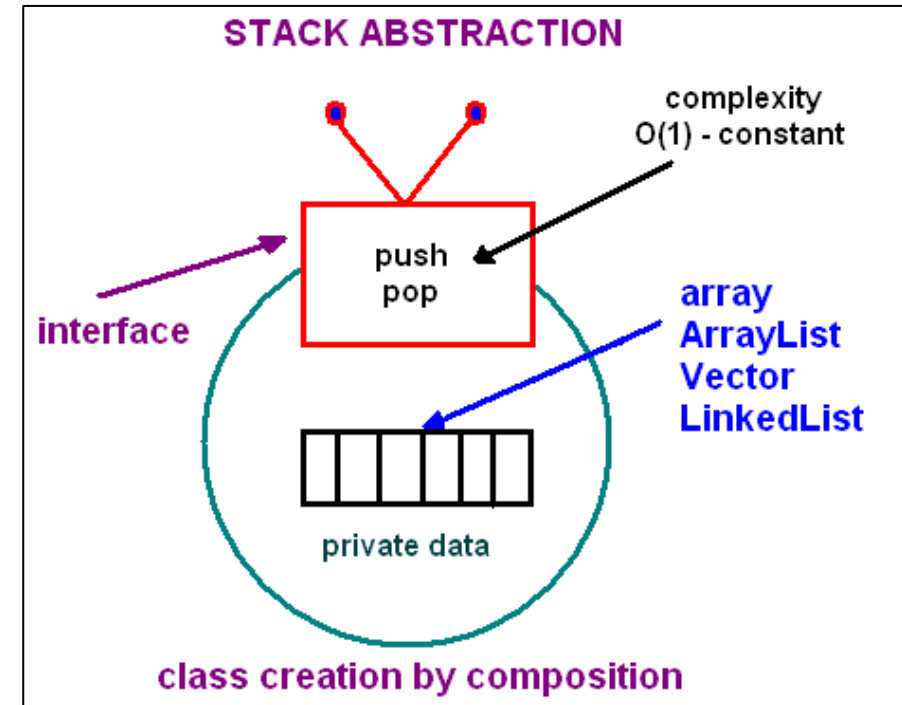
- **size()**: Returns the number of objects in stack
Input: none; *Output*: integer
- **isEmpty()**: Return a boolean indicating if stack is empty.
Input: none; *Output*: boolean
- **top()**: return the top object of the stack, without removing it; if the stack is empty an error occurs.
Input: none; *Output*: Object

```
1 package edu.colval.dsa.stack
2 /**
3  * This trait defines a mutable Stack ADT.
4  * @tparam A the type of data stored
5  */
6 trait IStack[T] { // Amine
7   /**
8    * Add an item to the stack.
9    * @param obj the item to add
10   */
11   def push(e: T)
12   /**
13    * Remove the most recently added item.
14    * @return the item that was removed
15   */
16   def pop(): T
17
18   /**
19    * Return the most recently added item
20    * without removing it.
21    * @return the most recently added item
22   */
23   def peek: T
24   /**
25    * Tells whether this stack is empty.
26    * @return true if there are no items
27    * on the stack, otherwise false.
28   */
29   def isEmpty: Boolean
30 }
```

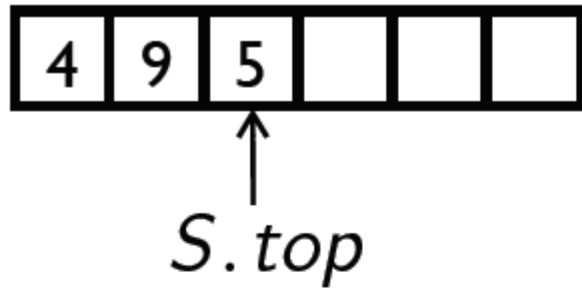
Implémenter un Stack :

Les alternatives

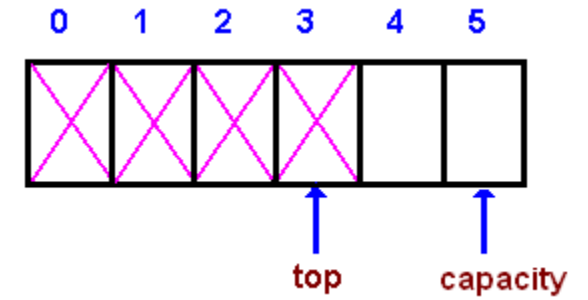
- Représentation contiguë :
 - Les éléments du Stack sont rangés dans un tableau
 - Un entier représente la position du sommet du Stack.
- Représentation chaînée :
 - Les éléments du Stack sont chaînés entre eux
 - Une référence du premier élément désigne le stack et représente le sommet de ce stack.
 - Un Stack vide est représentée par une référence NULL



Implémenter un Stack : Tableau ordinaire



```
PUSH(S, x)  
1  if S.top == S.length  
2      error "overflow"  
3  S.top = S.top + 1  
4  S[S.top] = x
```



```
STACK-EMPTY(S)  
1  return S.top == 0
```

```
POP(S)  
1  if STACK-EMPTY(S)  
2      error "underflow"  
3  else S.top = S.top - 1  
4      return S[top(S) + 1]
```

Implémenter un Stack :

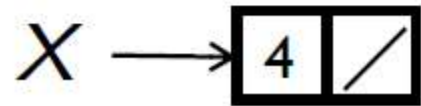
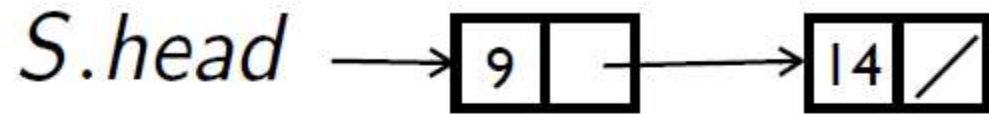
Tableau ordinaire – Version Scala

```
3 import scala.reflect.ClassTag
4
5 class MutableArrayStack[T: ClassTag](maxsize: Int)
6 extends IStack[T] {
7   private var top = 0
8   private var data = new Array[T](maxsize)
9
10  override def push(e: T) : Unit = {
11    if (top >= data.length) {
12      val largerArray = new Array[T](data.length * 2)
13      Array.copy(data, 0, largerArray, 0, data.length)
14      data = largerArray
15    }
16    data(top) = e
17    top += 1
18  }
```

```
20  override def pop(): T = {
21    assert(!isEmpty, "Pop called on an empty stack.")
22    top -= 1
23    data(top)
24  }
25
26  override def peek: T = data(top - 1)
27
28  override def isEmpty: Boolean = top == 0
29 }
```

```
1 package edu.colval.dsa.stack
2
3 object AppDriver extends App {
4   println("Stack AppDriver...")
5   val stackOfInt =
6     new MutableArrayStack[Int](10)
7   stackOfInt.push(12)
8   stackOfInt.push(5)
9
10  while (!stackOfInt.isEmpty) {
11    println(stackOfInt.pop())
12  }
13 }
```

Implémenter un Stack: Liste chaînée simple



PUSH(S, x)

- 1 $x.next = S.head$
- 2 $S.head = x$

STACK-EMPTY(S)

- 1 **if** $S.head == \text{NIL}$
- 2 **return** TRUE
- 3 **else return** FALSE

POP(S)

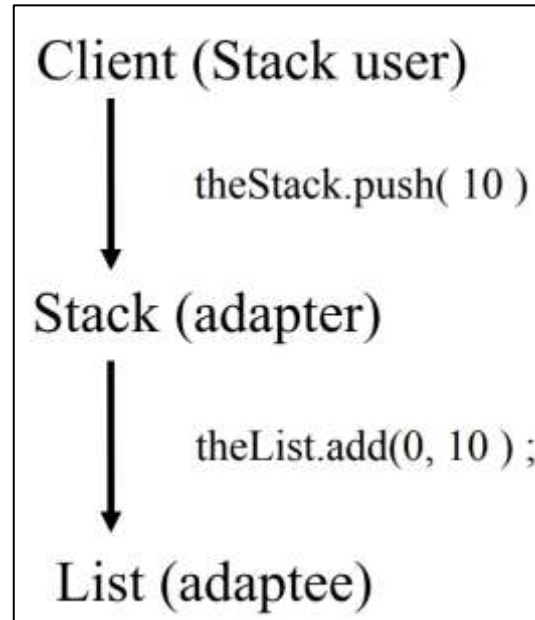
- 1 **if** STACK-EMPTY(S)
- 2 **error** "underflow"
- 3 **else** $x = S.head$
- 4 $S.head = S.head.next$
- 5 **return** x

Adapter une liste pour implémenter un Stack

- Deux alternatives :

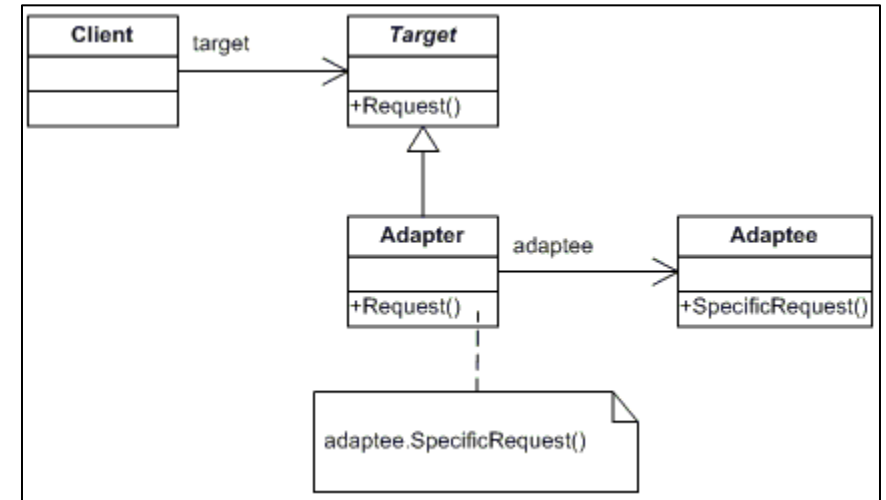
- Héritage :

- Stack hérite de la liste. Les méthodes de la liste sont protégées et donc accessibles uniquement par le Stack.



- Délégation :

- Patron Adapter
 - Le Stack est l'adapter
 - La liste est l'adaptée



<i>Stack Method</i>	<i>Singly Linked List Method</i>
<code>size()</code>	<code>list.size()</code>
<code>isEmpty()</code>	<code>list.isEmpty()</code>
<code>push(e)</code>	<code>list.addFirst(e)</code>
<code>pop()</code>	<code>list.removeFirst()</code>
<code>top()</code>	<code>list.first()</code>

Complexité des opérations d'un Stack

- Les opérations sur les piles sont toutes en $O(1)$.
- Ceci est valable pour les deux représentations proposées.

Data Structure	Time Complexity			
	Worst			
	Access	Search	Insertion	Deletion
<u>Array</u>	$O(1)$	$O(n)$	$O(n)$	$O(n)$
<u>Stack</u>	$O(n)$	$O(n)$	$O(1)$	$O(1)$
<u>Queue</u>	$O(n)$	$O(n)$	$O(1)$	$O(1)$
<u>Singly-Linked List</u>	$O(n)$	$O(n)$	$O(1)$	$O(1)$
<u>Doubly-Linked List</u>	$O(n)$	$O(n)$	$O(1)$	$O(1)$

Applications du Stack :

Appariement de parenthèses ($[]$, $()$ ou $\{\}$)

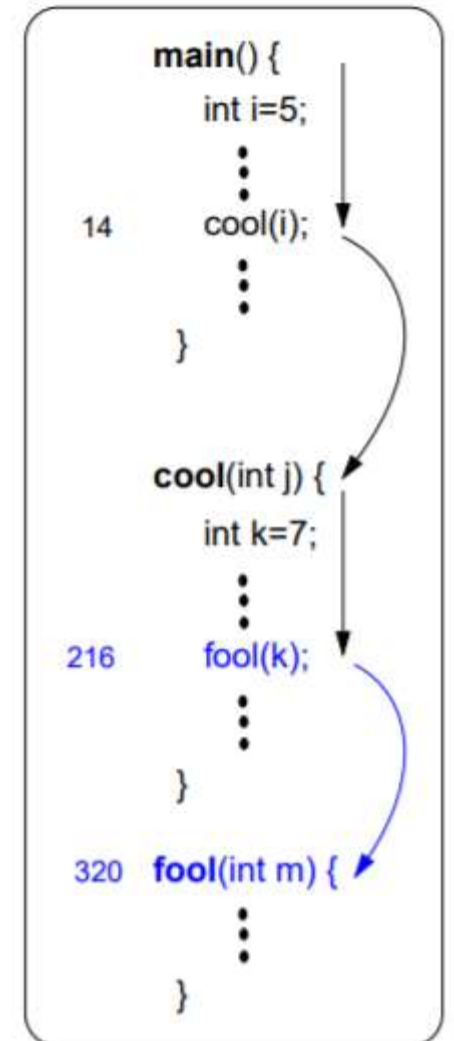
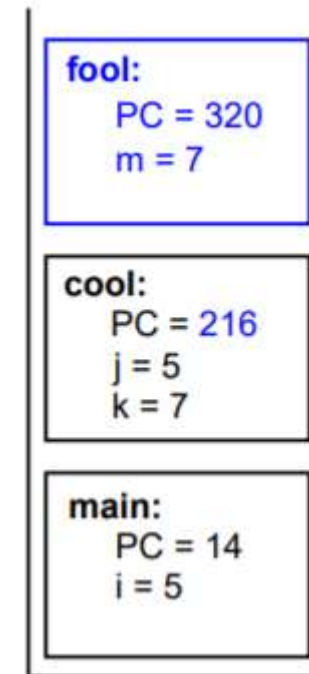
PARENTHESESMATCH(A)

```
1   $S$  = pile vide
2  for  $i = 1$  to  $A.length$ 
3      if  $A[i]$  est une parenthèse gauche
4          PUSH( $S, A[i]$ )
5      elseif  $A[i]$  est une parenthèse droite
6          if STACK-EMPTY( $S$ )
7              return FALSE
8          elseif POP( $S$ ) n'est pas du même type que  $A[i]$ 
9              return FALSE
10 return STACK-EMPTY( $S$ )
```

Applications du Stack :

Pile d'exécution d'un programme

- Les appels sont empilés.
 - Les paramètres de la fonction appelée
 - Les variables locales
 - L'adresse de retour à la fonction appelant
- Une fois une fonction terminée, son environnement d'exécution est dépilé.
- Algorithmes récuratifs consomme beaucoup la pile.



Applications du Stack :

Mot palindrome

- Le palindrome, aussi appelé palindrome de lettres, est une figure de style désignant un texte ou un mot dont l'ordre des lettres reste le même qu'on le lise de gauche à droite ou de droite à gauche, comme dans la phrase « Ésope reste ici et se repose » ou encore « La mariée ira mal » à un accent près.

```
public static boolean isPalindrome(String s) {  
    char nextChar;  
    String newS = s.toUpperCase();  
    StackOfChar stk = new StackOfChar();  
    QueueOfChar que = new QueueOfChar();  
    for (int j=0; j<newS.length(); j++) {  
        nextChar = newS.charAt(j);  
        if (('A'<=nextChar && nextChar<='Z')  
            || ('0'<=nextChar && nextChar<='9'))  
        {  
            stk.push( nextChar );  
            que.enqueue( nextChar );  
        }  
    }  
    boolean charsMatch = true;  
    while ( !que.isEmpty() ) {  
        charsMatch = charsMatch  
            && (que.frontChar() == stk.topChar());  
        que.dequeue();  
        stk.pop();  
    }  
    return charsMatch;  
}
```

Applications du Stack :

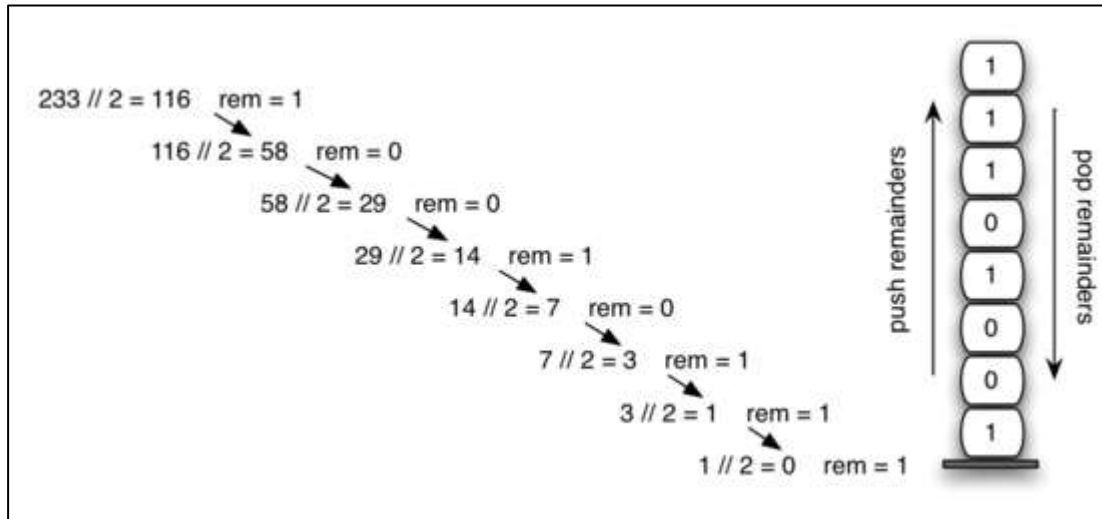
Évaluation d'une expression arithmétique

- Let's assume the below
 - Operands are real numbers.
 - Permitted operators: +, -, *, /, ^ (exponentiation)
 - Blanks are permitted in expression.
 - Parenthesis are permitted
- We will use two stacks
 - Operand stack: This stack will be used to keep track of numbers.
 - Operator stack: This stack will be used to keep operations (+, -, *, /, ^)

1. If the character is an operand, push it to the operand stack.
2. If the character is an operator,
 1. If the operator stack is empty then push it to the operator stack.
 2. Else If the operator stack is not empty,
 - If the character's precedence is greater than or equal to the precedence of the stack top of the operator stack, then push the character to the operator stack.
 - If the character's precedence is less than the precedence of the stack top of the operator stack then do **Process** (as explained above) until character's precedence is less or stack is not empty.
3. If the character is "(", then push it onto the operator stack.
4. If the character is ")", then do **Process** (as explained above) until the corresponding "(" is encountered in operator stack. Now just pop out the "(".

Applications du Stack :

Convertir un nombre décimal en binaire



```
void dec_to_bin(int number) {  
    stack<int> stk;  
    while(number > 0) {  
        int rem = number % 2;  
        number = number / 2;  
        stk.push(rem);  
    }  
    while(!stk.empty()) {  
        int item;  
        item = stk.top();  
        stk.pop();  
        cout << item;  
    }  
}
```

Applications du Stack :

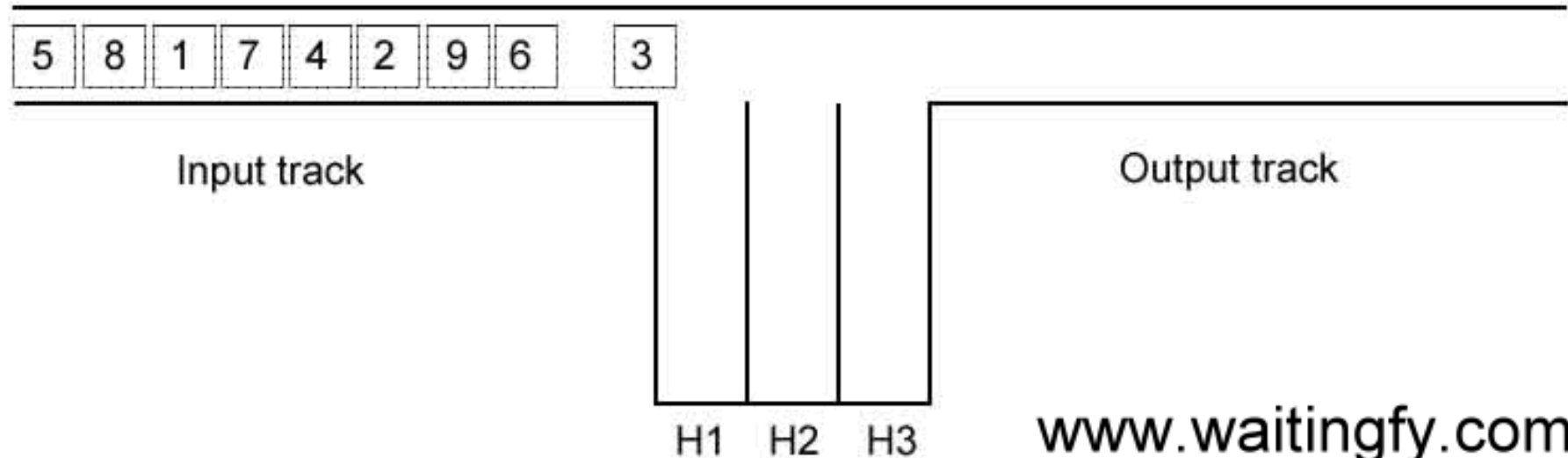
Convertir une expression infixe en postfixe

- Suppose Q is an arithmetic expression in infix notation. We will create an equivalent postfix expression P by adding items to on the right of P. The new expression P will not contain any parentheses.
- We will use a stack in which each item may be a left parenthesis or the symbol for an operation.
- Start with an empty stack. We scan Q from left to right.

<http://faculty.cs.niu.edu/~hutchins/csci241/eval.htm>

```
While (we have not reached the end of Q)
  If (an operand is found)
    Add it to P
  End-If
  If (a left parenthesis is found)
    Push it onto the stack
  End-If
  If (a right parenthesis is found)
    While (the stack is not empty AND the top item is
      not a left parenthesis)
      Pop the stack and add the popped value to P
    End-While
    Pop the left parenthesis from the stack and discard it
  End-If
  If (an operator is found)
    If (the stack is empty or if the top element is a left
      parenthesis)
      Push the operator onto the stack
    Else
      While (the stack is not empty AND the top of the stack
        is not a left parenthesis AND precedence of the
        operator <= precedence of the top of the stack)
        Pop the stack and add the top value to P
      End-While
      Push the latest operator onto the stack
    End-If
  End-If
End-While
While (the stack is not empty)
  Pop the stack and add the popped value to P
End-While
```

Applications du Stack : Rearranging railcars



www.waitingfy.com

Applications du Stack :

Parcours Pre-order d'un arbre

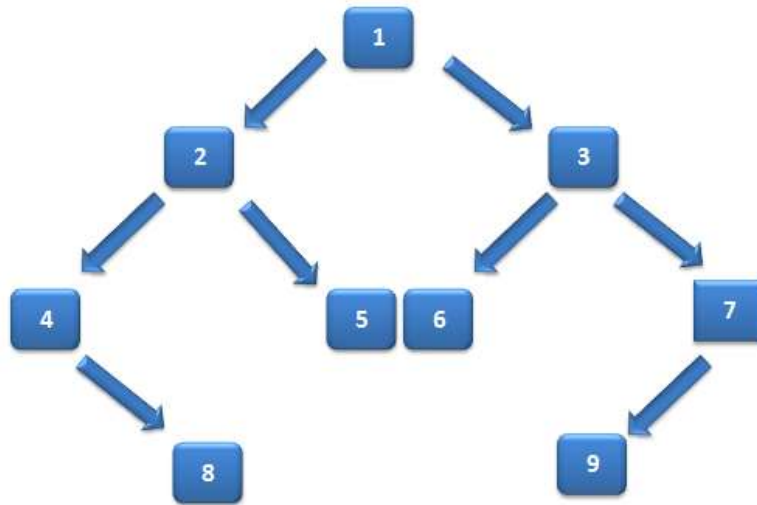


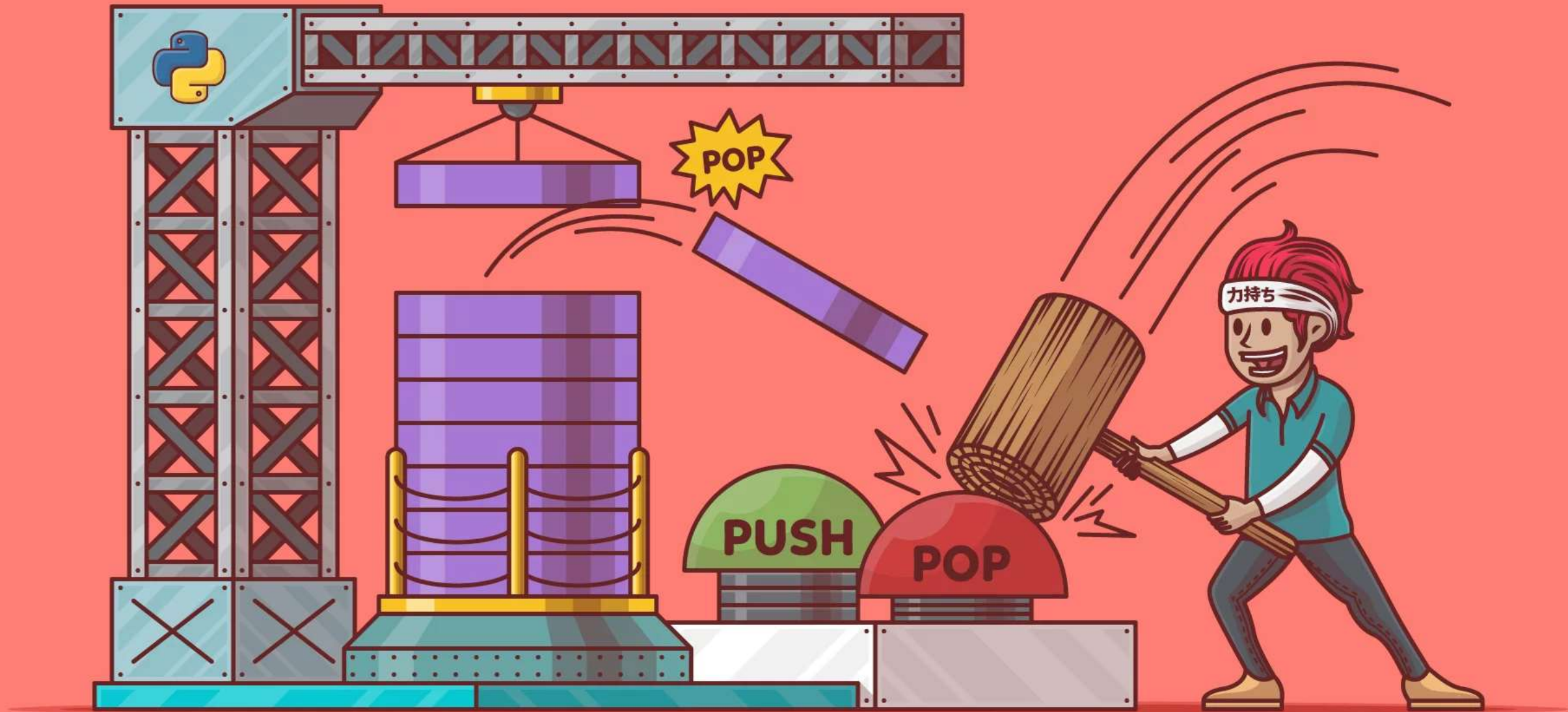
Fig: Binary Tree

Pre-order (Root, Left, Right) :
1, 2, 4, 8, 5, 3, 6, 7, 9
Post-order (Left, Right, Root)
8, 4, 5, 2, 6, 9, 7, 3, 1

Algorithm Preorder(tree)

1. Visit the root.
2. Traverse the left subtree, i.e., call Preorder(left-subtree)
3. Traverse the right subtree, i.e., call Preorder(right-subtree)

```
1 /**
2  * Iterative Pre-order traversal using single stacks
3  */
4 public void iterativePostOrder(Node root) {
5     Stack<Node> stack = new Stack<Node>();
6     Node curr = root;
7     for (;;)
8     {
9         if (curr != null)
10        {
11            if (curr.right != null)
12                stack.push(curr.right);
13
14            stack.push(curr);
15            curr = curr.left;
16        }
17        else
18        {
19            if (!stack.isEmpty())
20            {
21                curr = stack.pop();
22                // odd case, exchange curr and top element
23                if (!stack.isEmpty() && curr.right == stack.peek())
24                {
25                    stack.pop();
26                    stack.push(curr);
27                    curr = curr.right;
28                }
29                else
30                {
31                    System.out.print(curr+"\t");
32                    curr=null; //move to right or null
33                }
34            }
35            else
36                break;
37        }
38    }
39 }
40
```



Références

- <https://www.cs.cmu.edu/~adamchik/15-121/lectures/Stacks%20and%20Queues/Stacks%20and%20Queues.html>
- <https://www.bigocheatsheet.com/>
- <https://techdifferences.com/difference-between-stack-and-queue.html>
- <https://www.javatpoint.com/ds-array-implementation-of-stack>